

System Architecture for On-line Optimization of Automated Trading Strategies

Fabio Daros Freitas *

Receita Federal do Brasil
Pietrângelo de Biase 56/308
29.010-190, Vitória E.S., Brazil
freitas@computer.org

Christian Daros Freitas

Receita Federal do Brasil
Pietrângelo de Biase 56/308
29.010-190, Vitória E.S., Brazil
cfreitas@computer.org

Alberto Ferreira De Souza †

DI, UFES
Av. Fernando Ferrari, 514
29075-910, Vitória E.S., Brazil
alberto@lcad.inf.ufes.br

ABSTRACT

This work proposes a new automated trading system (ATS) architecture that supports multiple strategies for multiple market conditions through hierarchical trading signals generation employing h-signals, which are trading signals that are generated using other trading signals. The central idea of the proposed system architecture is to decompose the trading problem into a set of tasks handled by distributed autonomous agents under a minimal central coordination. We implemented the proposed ATS using a software architecture that employed a publish/subscribe communication model. In the current stage of development, we are able to run our ATS in back-test mode with moving-average crossover strategies on minute-by-minute market databases. We achieved very satisfactory performance results, processing 306.791 database rows representing more than two years of data in only 47 seconds.

Categories and Subject Descriptors

C.2.4 [Computer-Communication Networks]: Distributed Systems—*Distributed applications*; H.3.4 [Information Storage and Retrieval]: Systems and Software—*Distributed systems*; J.4 [Computer Applications]: Social And Behavioral Sciences—*Economics*

General Terms

Design, Algorithms.

Keywords

Automated Trading Systems (ATS), Algorithmic Trading, Stock Markets, System and Software Architectures.

1. INTRODUCTION

*Corresponding author.

†DI: Departamento de Informática;
UFES: Universidade Federal do Espírito Santo.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

WHPCF'13 November 18 2013, Denver, CO, USA

Copyright 2013 ACM 978-1-4503-2507-3/13/11

<http://dx.doi.org/10.1145/2535557.2535563> ...\$15.00.

The aftermath of 2008 financial crisis brought significant interest to *algorithmic trading* (AT) and in particular to its high-speed variant *high frequency trading* (HFT) [9, 17]. While there is ongoing debate of its merits and flaws, HFT is still experiencing steady growth and solely accounts for more than 50% of global equities and futures volumes and about 40% of global currency volumes [8, 10, 22, 13, 12, 17].

Institutions and individuals buy and sell financial assets for many reasons, including keep controlling interest in companies, receive dividends, and to profit from trades. In this context, we can define a *trade* as the realization, or *execution*, of at least one entry position in an asset (e.g. a stock) followed, some time latter, by an exit position. To enter a position, one goes *long* (buy) or *short* (sell short), while the opposite operation (sell or buy to cover) is required to partially or fully exit that position [19]. A *trading strategy* can be defined as a systematic sequence of actions taken with the goal of effectively executing trades, typically consisting of a set of entry, exit, and order execution rules. *Automated trading strategies* are those that are partially or totally implemented by computer programs without discretionary human interventions [15].

HFT and other algorithmic trading strategies typically use a high speed computational infrastructure to receive and store market data, analyze it, and send buy and sell orders to the market according the implemented investment strategy. The coupling of computer programs implementing these trading strategies and computational and communication infrastructures make the *automated trading systems* (ATS), that are the main engine of many quantitative investment funds administered by various financial institutions [6].

The various tasks a successful ATS must handle include receiving, cleaning, evaluating, and archiving of incoming quotes and other market data; on-line econometric analysis and other model-driven computations for evaluating trading strategies; planning of order properties and execution for implementing required positions; sending orders to market and receiving status data and other market information to control its executions; on-line portfolio management; updating of performance metrics; and risk management based on current asset allocations [2].

In many of these tasks, in particular those related with trading signals generation and orders execution control, the ATS is required to adapt as fast as possible to temporary market conditions. This adaptive behavior may be hard to accomplish, mainly due to changing environments that increase the risk of effectiveness of trading strategies, which is frequently known only after performance degradation is verified [15].

Trading strategies and trading rules generation are a well covered topic in academic and practitioner literature [4, 19, 2]. The optimizing approach to accomplish adaptiveness with these tra-

ditional methods is to *back-test* the strategy by simulating its behavior on known past data and finding a set of parameters values that produce the best result according some performance measures. Although back-testing is a fundamental tool in designing and developing trading strategies, it shows many limitations when used solely as adaptation method to market changes. These limitations are related to the off-line and frequently time consuming nature of back-testing, in which huge volumes of data must be examined. Adaptive trading and execution strategies have been proposed in literature [11, 3, 5, 16], and despite they are a promising direction in addressing the off-line issues of traditional back-testing, its application domain (e.g. trading frequencies) is restricted by algorithm's computational performance.

An alternative to this traditional approach for ATS adaptation to market changing conditions is to explore system architectures that allow for simultaneously running various instances of trading strategies, whose results are evaluated in on-line fashion to ultimately produce investment decisions. In spite of trading system architectures coverage in literature has been increasing in recent years, it is mainly focused on describing system's building blocks rather than on optimization and implementation issues [18, 17, 2].

Raudys [20] presented a two-stage method for optimizing portfolios of many ATS based on its time series of profits and losses, and a two-layer trading design architecture that was employed at the automated trading firm that provided the data used in his study. The firm trades in U.S. and European futures, including stock indices, energies, metals, commodities, energies, interest rate products, and FX products. The ATS were directly used in the first layer of the architecture, while the robots outputs were fused by calculating its weights in the second layer. Clustering of input variables, in order to express the covariance matrix in a block-diagonal way, and performing decision making as a two-stage multi-agent system are proposed as a solution to enhance the effectiveness of the portfolio weights calculation. His experimental results with real financial data from 2002 to 2012 confirm the effectiveness of the proposed approach, beating the 1/N and Markowitz model fusion strategies.

Koors and Page [15] evaluated the state of the art of trading system architectures examining the literature and commercial products. The authors characterized the *simulation* and *interactive* running modes of ATS, when the systems operate in back-test mode or in live trading mode, respectively; and evaluated how basic architectures for ATS respond to the problem of adaptation to market changing conditions. Architectures that support multiple strategies for multiple market aspects are pointed as suitable alternatives, and the related problem of strategy selection that arise in this approach is addressed. A new hierarchical software architecture containing simulators as an integral component is proposed. In this approach, basic strategies adjusted on special aspects of the application domain are each assigned a separate simulator, determining success curves, performance indicators, and action histories for their simulated behavior. These intermediate results are constantly updated and analyzed by superior evaluation strategies on several hierarchical layers. Each evaluation strategy imitates, combines, or synthesizes the behavior patterns of the strategies of the subordinated layer against its own simulator. Finally, in interactive mode, the most superior evaluation strategy is connected with the real environment by its simulator.

In this work, we propose a new ATS system architecture that supports multiple strategies for multiple market conditions through hierarchical trading signals generation employing *h-signals*. We define a *h-signal* as a trading signal that is generated using market data and also other trading signals as inputs. This architecture was conceived through decomposing the trading problem into a set of

tasks that are handled by distributed autonomous agents under a minimal central coordination. As a result, we can obtain a distributed ATS implementation that can run on various machines, possibly exploring high performance hardware features suitable for executing specialized trading strategies in order to efficiently trade at various market conditions.

The remainder of this paper is organized as follows. After this introduction, Section 2 presents the proposed ATS system architecture and Section 3 outlines the software architecture we employed to implement our ATS. In the sequence, Section 4 presents the current status of work, reporting some achieved results, and Section 5 discusses some relevant topics in the context of this work. Our conclusions and directions for future research are presented in Section 6.

2. SYSTEM ARCHITECTURE MODEL

Trading strategies rely on systematic application of a set of rules to enter a trade and another set of rules, which are not necessarily the opposite of the former, to exit the trade expecting to achieve a profitable return [19]. Trading systems implement one or more trading strategies, and automated trading systems, or ATS, are those that are partially or totally implemented by computer programs without discretionary human interventions [15].

An ATS can be viewed as a set of system components that periodically receives market data in its inputs, computes econometric and decision making models, and outputs buy and sell orders that are routed back to the market. Data received from the market regarding the executions of these orders provide a feedback input to the system.

A successful ATS must have two operation modes: (i) *back-test mode*, when the system runs in simulation mode using previously archived inputs and is employed in the designing, tuning, and testing of trading strategies; and (ii) *live mode*, or *interactive mode* [15], when the system is connected to the market and is continually receiving new data and ready to produce and route new orders.

This section describes a new ATS system architecture that supports multiple strategies for multiple market conditions through hierarchical trading signals generation employing *h-signals*. The trading problem was decomposed into the following basic parts and respective associated tasks: (i) *signal generation*: implements investment advising tasks by evaluating market data relative to a set of assets in search of trading opportunities according to predefined entry and exit rules, producing trading signals that represent the found opportunities; (ii) *trading*: observes trading signals and apply sets of predefined trading rules and constraints to produce decisions to hold or liquidate positions in a set of assets; and (iii) *operation*: observes demands for holding or liquidating positions in assets and apply predefined rules to convert them in buy and sell orders, send these orders to the market, monitors its execution, and report the executed positions.

Decomposing the trading problem in this way allow us to define the *Signal Generator* (or *Signal* for short), *Trader*, and *Operator* autonomous agents that implement, respectively, the above described basic parts and tasks for an ATS. Signals and Traders agents compound together to form *Strategy* virtual agents, that are responsible for implementing the defined trading strategies that will be operated by Operator agent. Here, a *virtual agent* is a representation of the joint operation of a group of agents. The proposed ATS system architecture is presented in Figure 1 as a two-level model. The high-level model, or *trading level*, is presented in Figure 1a and belongs to a category of ATS architectures that supports multiple basic strategies with separated trading accounts and superordinated risk management, as described in [15]; while the lower-level model,

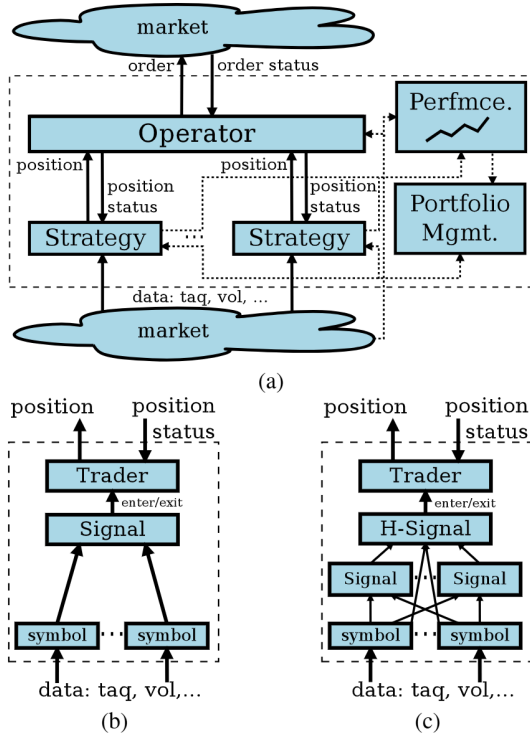


Figure 1: ATS system architecture (a). Strategy agent architecture with signal generator (b), and with h-signal generator (c).

or *strategy level*, is presented in Figures 1b and 1c and is responsible for producing position demands used by the trading level.

In the trading level of the architecture, which is presented in Figure 1a, Strategy virtual agents receive market data, including trades and quotes (TAQ) data that records traded and quoted (in the books) prices and quantities, and volume data; and compute demands for holding or liquidating positions according to strategies' entry and exit rules. In this context, a *position* on a determined asset, or *symbol*, can be defined as the desired financial volume, or number of shares or contracts, to be held or liquidated depending on position's type. In equities trading, for example, position types are typically an "Enter Long", "Exit Long", "Enter Short", or "Exit Short" [19].

The Operator agent then receives these position demands and is responsible for converting them into orders using symbol's TAQ and other available market data, and to route these orders to the market through direct market access (DMA) to exchanges or using brokers' electronic platforms. In this process, the Operator agent may implement methods for minimizing market impact and trading risk [14]. The Operator agent is also responsible for monitoring orders execution status and report positions status back to Strategy virtual agents.

During each trade's course, the Strategy virtual agents maintain performance data that are used by Performance agent (labeled "Perfmc." in Figure 1a) for calculating equity curves and performance metrics, and by Portfolio Management agent (labeled "Portfolio Mgmt." in Figure 1a) for calculating strategies and investment risks, and portfolio models for optimizing the allocation of funds to each strategy. Portfolio Management agent also uses data produced by Performance agent to perform its tasks.

In the strategy level of the architecture, which reflects our approach for decomposing trading strategies into signal generation and trading (see Figures 1b and 1c), the Signal and *H-Signal* agents act as investment advisors that produce recommendations to enter or exit on symbols, which then will be considered by a Trader agent to be used for generating demands for positions on those symbols.

Signal agents implement entry and exit trading rules for one or more symbols using their market data and sending respective trading signals to its outputs, as showed in Figure 1b. When operating with only one symbol, the Signal agent observes only that symbol's market data, compute its trading rules, and generate its respective trading signals. For example, a moving-average Signal agent operating on only one symbol reads that symbol's past series of prices, calculate the desired moving averages, and generate trading signals according the crossing patterns of the prices and the moving averages. When operating with more than one symbol, a Signal agent calculates individual trading signals and select those that will be sent to its output. For example, a moving-average Signal agent operating on various symbols must calculate each individual trading signal according the crossing patterns of prices and moving averages, select the better ones according some evaluation criteria, and send them to its output.

H-Signal agents combine trading signals from one or more Signal agents to synthesize new trading signals, or *h-signals*, that are sent to its outputs, as showed in Figure 1c. They also have access to symbols' market data that can be employed to implement entry or exit *filter rules*, which adds additional information to produce more accurate and reliable trading signals [19] (Signals agents can also use symbols' market data this way). It is important to note that the h-signal definition is recursive, and one can have a H-Signal agent that combines other H-Signal agents outputs (not shown in Figure 1c). This can be viewed as a high-hierarchical trading signal generation, which adds virtually unlimited capabilities for trading strategies representation in the architecture. For example, to implement an adaptive ATS that optimizes a moving-average trading strategy on one symbol, one can have various moving-average Signal agents, each one calculating a moving average of respective length, that are combined by a H-Signal agent that employs a selection criteria, for example the trading strategy accuracy method (TSA) [1], to select, at each moment, the Signal agent (i.e. the moving average length) that better adapts to current market condition.

Finally, the Trader agent connected to the Signal (Figure 1b) or top H-Signal (Figure 1c) agent receives the produced entry or exit trading signals, compute strategy's model parameters, including available capital, trading stops, and other strategy's risk management parameters, and apply decision making algorithms to produce demands for holding or liquidating positions in its output. While receiving positions status generated by Operator agent (see Figure 1a), Trader agent updates strategy's model state with the symbols' executed prices and quantities, and generates performance data that will be used by Performance and Portfolio Management agents.

The above presentation of our ATS system architecture considered the live operation mode, when the system is connected to the market and ready to produce and route new orders. The back-test operation mode, when the system runs in simulation mode and is employed in designing, tuning, and testing trading strategies, can be attained in two ways: simulating order status information, or simulating position status information (see Figure 1a). These status information refer to the execution details of the orders, which are generated by broker or exchange platforms, or to the execution details of the positions, which are generated by the Operator agent.

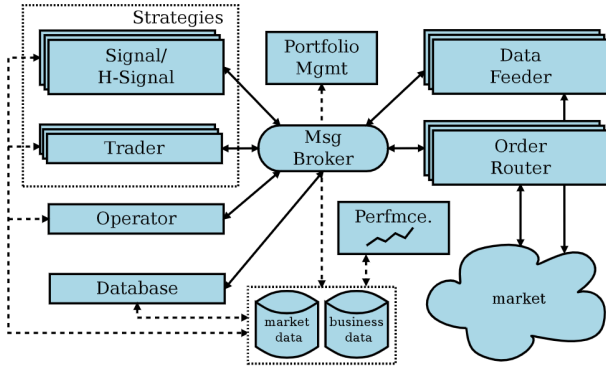


Figure 2: ATS software architecture.

Having these two levels of simulation is a very interesting feature, since it allows running the back-test simulations in a more precise and complete mode that includes the algorithms for minimizing market impact and trading risk, when simulating order status information at the Operator agent; or running the back-test simulations in as-fast-as-possible mode, when simulating position status information directly at the Trader agent.

3. SOFTWARE ARCHITECTURE MODEL

Our proposed ATS system architecture model was presented in Section 2, through decomposing the trading problem into a set of tasks regarding the Signal, H-Signal, Trader, Operator, Performance, and Portfolio Management distributed autonomous agents. The present section presents the software architecture model that we used for implementing the ATS system architecture, which employed a publish/subscribe communication model, or *pub/sub* for short.

In the pub/sub communication model, the provider of the information is called a *publisher*, the consumer of the information is called a *subscriber*, and a message broker acts as an intermediary between the two parties. Processes indicate their interest in receiving messages of a certain type becoming a subscriber of that message type, and when other processes publish messages of that type, thus becoming a publisher of that message type, all the subscribers receive a copy of the message [21]. Pub/sub model is a well-suited choice for implementing systems of autonomous agents [7] and exhibits several advantages, including decoupling of the provider of information from the consumers of that information, absence of specific requirements for the messages recipients, asynchronous message reception, and applications that are not tied to particular partners. Our proposed software architecture model is presented in Figure 2, which shows the main system modules and its respective message exchange paths (in solid lines) and storage accesses (in dotted lines).

The basic operations of the system modules of Figure 2 are described as follows:

- **Data Feeder:** this module is responsible for supplying market data in real time to the system. The fundamental data feeds are the trades and quotes (TAQ) data of the various symbols of interest, but other kinds of data may be supported. The software architecture supports multiple data vendors through running various instances of the Data Feeder module and connecting each one to a specific data vendor platform. Once data is received from a data vendor, their respective messages are published by its Data Feeder module.

- **Database:** market and business data are stored in different specialized structures in the system's databases. Market data are those that are published by Data Feeder modules (e.g. TAQ data) or derived from them, and are used as input to all the trading algorithms of the system. They require high performance access and are stored in specialized structures. Database module complements the system's main market data intake by subscribing to all the messages that are published by Data Feeder modules and storing their respective data in the system's market database upon its reception; when it also publishes trading tick messages announcing that new market data has arrived and is ready to be used. Business data are those that record the system's definitions, parameters, internal states, operations, and other events. Basically, it consist in all data needed for system operation except market data. Database module maintains business data by subscribing to operational messages that are published by various other modules.

- **Signal/H-Signal:** these modules are responsible for applying the entry and exit rules (algorithms) and to produce the trading signals that are part of the trading strategies of the ATS, as described in Section 2. A Signal module implements a specific entry/exit algorithm, and the simultaneous execution of various algorithms can be obtained by simultaneously running various Signal modules. Executing various instances of the same algorithm can be obtained by running various instances of the same Signal module. As described in Section 2, the H-Signal modules can aggregate various Signal modules and thus implement more sophisticated algorithms. In live mode, Signal and H-Signal modules subscribe to trading tick messages that inform the arrival of new market data and act upon its reception. The typical action is reading market databases, computing entry/exit trading signals, and publishing its respective messages. In back-test mode, Signal and H-Signal modules run in stand alone mode continuously reading input data from market databases, computing entry/exit trading signals, and saving its respective data into signal databases.

- **Trader:** these modules are responsible for complementing the trading strategies of the ATS by using the trading signals produced by Signals and H-Signals modules, as described in Section 2. Distinct trading strategies can be implemented by parameterizing instances of a general Trader module or by implementing specific Trader modules. A Trader module observes the outputs of only one Signal/H-Signal module and runs its trading algorithms with the goal of utilize the trading signals produced by its observed module, seeking to generate profitable position signals, which are demands for holding or liquidating positions. In live mode, the Trader module keeps aware of new trading signals by subscribing to the messages published by the observed Signal/H-Signal module and acts upon its reception. The typical action is reading market databases, computing the trading strategy algorithms and internal states, including available capital, stops, and other trade parameters; and publishing position signals messages. In back-test mode, the Trader module runs in stand alone mode continuously reading input trading signals from signal databases created by its observed Signal/H-Signal, computing the trading strategy algorithms and internal states, producing positions signals and simulating its executions, and logging its respective data for performance reporting.

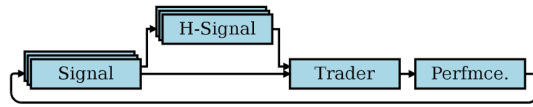


Figure 3: Schema for the ATS software architecture's trading core modules running in back-test mode.

- Operator:** this module is responsible for receiving position signals and convert them to orders using symbol's TAQ and other market data. The Operator module observes the outputs of Trader agents and run its algorithms with the goal of converting demands for holding or liquidating positions in a set of orders that will be routed to the market. In live mode, the Operator module subscribes to position signal messages that are published by Trader modules and acts upon its reception. The typical action is reading the desired position type (entry or exit, long or short) and size (number of shares or contracts), reading market databases for calculating the orders necessary to implement that position, and publishing order messages that will be received by Order Router agents. In this task, the Operator module may employ algorithms for minimizing market impact and trading risk, and suitable slicing and timing of the orders. The Operator module subscribes to messages regarding order executions status that are published by the Order Router modules, and publishes position status messages when the respective positions are filled or canceled. In back-test mode, the Operator module can simulate the orders executions and publish position status messages.
- Order Router:** these modules are responsible for routing orders to the market and monitoring its executions. They subscribe to the order messages published by the Operator module and route them to the market through direct market access (DMA) to exchanges or using brokers' electronic platforms. The Order Router modules publish order status messages regarding the orders' execution status at the arrival of these information from the market. The proposed ATS software architecture supports connections to multiple DMA or broker's systems through running multiple instances of the Order Router module, thus allowing simultaneous operation in various markets.
- Portfolio Management:** this module implements a global vision of all running strategies. It can monitor strategies risks and calculate portfolio models to allocate capital to strategies accordingly, thus minimizing the global risk of the investment project. The Portfolio Management module accesses the business databases for reading strategies data and setting strategies parameters.
- Performance:** this module calculates all performance measures for all strategies running on the ATS. It accesses the market and business databases for reading market and strategies data regarding the performance of strategies and investments.

In back-test mode, during the designing, tuning, and testing of trading strategies, the trading core modules Signal, H-Signal, and Trader, which implement the strategy level agents of the ATS system architecture of Section 2 (see Figures 1b and 1c), all run in sequential mode as show in Figure 3 and described as follows.

The simpler case involves a strategy that employs only one Signal generator module. In this case, the Signal module is run on the

desired input data (symbols, market data, and timestamps range) and the produced trading signals are stored in the Signal generator's local databases. After this, strategy's Trader module is run using Signal generator's trading signals as inputs and stores its outputs (i.e. trade records consisting of entry and exit executed positions) in local databases that will be read by Performance module (labeled "Perfmce." in Figure 3) which, in turn, calculates all strategy's performance measures. This process is repeated until all strategy's parameters are adjusted in a satisfactory way.

A more complex case involving a two-level strategy that employs various Signal generators and one H-Signal module is described as follows. First, all Signal modules are run individually as described above. After this, strategy's H-Signal module is run using the Signals' stored trading signals and produce strategy's final hierarchical trading signals. In the sequence, strategy's Trader module is run using H-Signal's trading signals as inputs and stores its outputs in local databases that will be read by Performance module which, in turn, calculates all strategy's performance measures.

Extreme cases involve a N-level strategy employing various Signal and H-Signal generators that are sequentially run according its hierarchically organized topology until the topmost H-Signal module is reached. This H-Signal module, in turn, will produce strategy's final highly-hierarchical trading signals that will be processed by Trader and Performance modules in the same way as described above.

Finally, it worth to note that Data Feeder and Order Router modules were not mapped into the system architecture of Section 2 (see Figure 1). This omission is because they are basically operational modules that lie inside the "market" cloud in the context of the system architecture represented in Figure 1.

4. STATUS OF WORK

This session reports our current works in implementing the ATS software architecture presented in Section 3. Our research has been conducted at the finance research group (CADF) of the High Performance Computing Lab (LCAD) of Federal University of Esp rito Santo (UFES), Brazil.¹ We are implementing the modules of the ATS software architecture using C language in Linux operating system (Ubuntu 12.04), adopting Carnegie Mellon University's Inter-Process Communication (IPC) pub/sub framework² [21] and SQLite software library³ for the market and business databases implementation.

Initially, our strategy was experimenting with the architecture to observe its performance properties while running all modules on a single machine before running them in a distributed environment. In this context, our choice was to use minute-by-minute market data, thus limiting our maximum trading frequency to one minute, and run system modules on a standard PC machine with Intel Core 2 Extreme Quad-Core Processor QX6000 (8M Cache, 2.40 GHz, 1066 MHz FSB), 4 GB RAM (DDR-2 1066 MHz), and SATA hard disks.

We conducted some simple empirical tests to access the performances of IPC framework and SQLite library in this environment, mainly because these libraries are critical to the system's modules overall performance.

We tested the throughput of IPC framework by having a module that published 10.000 messages at a rate of one message per millisecond and another module that subscribed and received all these messages. We then varied the message's sizes measuring the

¹<http://www.lcad.inf.ufes.br>

²<http://www.cs.cmu.edu/~ipc>

³<http://www.sqlite.org>

Message size in bytes	Total bytes	Elapsed time	Throughput	
			Maximum	Measured
50B	500KB	0:00:11.28	50 KB/s	44.3 KB/s
5KB	50MB	0:00:11.60	5 MB/s	4.3 MB/s
50KB	500MB	0:00:12.18	50 MB/s	41 MB/s
500KB	5GB	0:00:51.74	500 MB/s	96 MB/s
5MB	50GB	0:09:22	5 GB/s	89 MB/s
50MB	500GB	1:29:21	50 GB/s	93 MB/s

Table 1: IPC throughput for publishing 10.000 messages of various sizes.

Operation	Number of records	Elapsed time	Estimated throughput
Writing	306.791	0:38.05	8K records/sec.
Reading a day	442	0:00.02	22K records/sec.
Reading a month	7962	0:00.39	20K records/sec.
Reading a year	101.640	0:05.00	20K records/sec.

Table 2: SQLite throughput for a TAQ database file with 306.791 records.

total elapsed time to receive all the 10.000 messages and calculating the obtained throughput by dividing the sum of all messages sizes by the total elapsed time. We also calculated the theoretic maximum throughput by dividing the same sum by 10 seconds (10.000 messages $\times$ 1 millisecond = 10 seconds). The obtained results are shown in Table 1, where each line reports one test result for the respective Message size, and the columns Total bytes, Elapsed time, and Throughput Maximum and Measured presents the calculated and observed variables. As Table 1 shows, message sizes near 50KB approximated the maximum throughput reasonably well. As a consequence, we designed all message types of the ATS architecture to remain below the 50KB size.

The SQLite library tests were conducted as follows. We defined the following schema

```
CREATE TABLE IF NOT EXISTS trade
(time INTEGER NOT NULL, timestamp TEXT NOT NULL,
prc REAL, qty INTEGER, vol REAL, PRIMARY KEY(time))
```

and created a TAQ database file by loading the 306.791 minute-by-minute recorded trades of the PETR4 Petrobras – PN stock in the Brazil’s BOVESPA stock exchange, while computing writing (loading) and reading times spent. The obtained results are presented in Table 2. As Table 2 shows, SQLite performed very well for our minute-by-minute setup, achieving a writing throughput of 8K records/second and a reading throughput of approximately 20K records/second in various situations. As the results of Tables 1 and 2 show, IPC framework and SQLite library fitted very well to our performance needs in the current stage of the ATS architecture development.

We also tested our ATS software architecture in back-test mode according the methods described in Section 3 and showed in Figure 3. In this task, we run a moving-average crossover strategy on the same minute-by-minute TAQ database file containing 306.791 rows that spanned for more than two years, and it took only 47 seconds to run. This was a very satisfactory result since it shows that it is possible to run the system in back-test mode in reasonable times while employing standard technical indicators.

The research and development of the modules of the ATS software architecture of Section 3 is an ongoing work, and in the current stage of development we have finished the first versions of

<i>Black Box</i> component	Implemented by ATS agents
Alpha Model	Signal, H-Signal, and Trader
Risk Model	Trader and Portfolio Management
Transaction Cost Model	Trader and Operator
Portfolio Construct Model	Portfolio Management
Execution Model	Operator

Table 3: Mapping between *Black Box* model and our ATS system architecture.

the Signal, H-Signal, Trader, Database, Data Feeder, Performance, and Order Router modules, enabling us to receive market data, routing orders and run trading strategies in back-test mode. The next development steps include the implementation of the Operator module, when the ATS system will be able to operate in live mode, and the Portfolio Management module, that will enable us to run multiple strategies simultaneously.

5. DISCUSSION

Designing, implementing, and running a successful ATS is recognized as a nontrivial task, and referred as “an arms race drawing on the skills of top computing professionals” [18, 2, 17].

Narang [17] introduced the *Black Box* model in 2009 and it became a widely accepted reference model for ATS system components architecture [18]. The mapping between the components of the *Black Box* model and the agents of our proposed ATS system architecture of Section 2 (see Figure 1) is presented in Table 3. As Table 3 shows, each row lists the functional correspondence between a *Black Box* system component and the agents of our ATS system architecture, showed in the “Implemented by agents” column.

Alpha Model is concerned with entry and exit rules definition and trading strategies construction for operating with these rules. These functionalities are implemented in the strategy level of our ATS system architecture (see Figures 1b and 1c) by the Signal, H-Signal, and Trader agents, which we use for decomposing the trading strategy problem in signal generation and trading subproblems.

Risk Model deals with avoidance of risk or loss reduction, and also with the intentional selection and sizing of exposures to improve the quality and consistency of returns. We implement these tasks in our ATS system architecture with the Trader and Portfolio Management agents. Trader agents deal with handling of stops and other loss avoidance mechanisms that are applied in the course of individual trades, while Portfolio Management agent addresses the joint risk of all trading strategies through optimizing risk portfolio models.

Transaction Cost Model is backed by the idea that it costs money to trade, which means that one should not trade unless there is a very good reason to do so. The main costs that are dealt here are the commissions and fees, that are paid to brokers and exchanges; the price slippages, that are the changes in the symbol’s prices between the time the system decides to transact and the time when the order is actually at the exchange for execution; and the market impact, that is the effect that the orders can cause in the prices themselves, typically in the opposite direction of the trade (buy orders rise the prices and sell orders low the prices). These functionalities are implemented in our ATS system architecture by the Trader and Operator agents. Trader agents deal with commissions and fees by calculating its costs before and after entering a trade, and also with price slippages by calculating its expected values; while Oper-

ator agent can handle market impact through employing execution algorithms like VWAP and TWAP [14].

Portfolio construct Model deals with the problem of choosing what portfolio to own. The decision to allocate funds on respective strategies is based on a balancing of considerations of expected return, risk, and transaction costs. Our ATS system architecture handles this problem with the Portfolio Management agent, that calculates strategies and investments risks, and portfolio models for optimizing the allocation of funds to each strategy.

Execution Model deals with the problem of order execution algorithms that determine the way in which systematic execution of positions are carried on. Among the various choices to decide on are the aggressive versus passive execution, larger versus smaller orders, and also where to send an order for execution. In our ATS system architecture, all these tasks are virtually handled by the Operator agent, albeit some specific tasks are strictly handled by the Order Router modules of the ATS software architecture of Section 3 (see Figure 2), that are not directly mapped into the ATS system architecture.

Koors and Page [15] proposed a new hierarchical software architecture containing simulators as an integral component. The authors highlighted the potential of their approach describing how evaluation strategies imitates, combines, or synthesizes the behavior patterns of basic strategies of the subordinated layer. For this purpose, the evaluation strategies of a layer rate the strategies of the subordinated layer in parallel and then independently carry out their own trading decisions based upon the subordinated strategies. The strategy level of our ATS system architecture, composed by the Signal, H-Signal, and Trader agents, implements such a hierarchical architecture in our system (see Figures 1b and 1c of Section 1).

We strongly believe that our architecture is able to implement all examples of strategies' behavior on the evaluation level that were reported in Koors and Page's work, mainly by exploring the Signal/H-Signal hierarchical relationship. We also expect a better performance of our system in this task, mostly in the back-test mode because our strategy level works with position execution simulation and not with the more complex problem of order execution simulation. Since back-testing is frequently used in a comparative way while developing, tuning, and testing strategies through benchmarking results of many runs, we believe that this approximation (using position execution instead of order execution) brings no drawbacks and allows the system to better cope with the performance needs of the as-fast-as-possible simulation mode.

Financial markets are highly complex and fast adaptive systems that result from the actions of a multitude of participants. At the system architecture level, the main challenges in designing a successful ATS involve tackling the problem of adaptation to market changing conditions. Trying to cope with all market phases with the same single strategy is an imprudent attempt, since the strategy has to incorporate too many degrees of freedom to consistently prove successful during back-testing [15]. As a consequence, this approach leads to over-fitted systems that fail when performing with data that was not used in back-test or, in live mode, when the market starts behaving in ways not seen in the back-test. Our proposed ATS system architecture tackles with this problem by supporting various Strategy virtual agents (composed by Signal/H-Signal and Trader agents) to simultaneously run and explore different market aspects, enabling our ATS to respond accordingly to different market conditions.

At the implementation level, the main challenges in running a successful ATS involve tackling the problem of latency. Many investment opportunities arise from temporary market conditions, allowing profitable exploration of market inefficiencies. Strategies

that explore statistical arbitrage or market microstructure using inventory models are some successful examples [2]. Albeit these market inefficiencies can occur at various frequencies, fast detection and response are key elements to successfully trading even on the lower frequencies opportunities, and critical for trading on higher frequencies. Our proposed ATS software architecture tackles with this problem by employing a low-latency pub/sub communication model to support the autonomous agents model implementation. Among the various advantages of this approach, the asynchronous message reception and the loose coupling of the system's components allow performance-critical agents to run on specialized machines for attaining fast and responsive algorithm implementations.

6. CONCLUSION

In this work, we propose a new automated trading system (ATS) architecture that supports multiple strategies for multiple market conditions through hierarchical trading signals generation employing *h-signals*. In this context, we define a *h-signal* as a trading signal that is generated using market data and also other trading signals as inputs.

The central idea of the proposed ATS system architecture was decomposing the trading problem into a set of tasks that are handled by distributed autonomous agents under a minimal central coordination. As a result, we can obtain a distributed ATS implementation that can run on various machines, possibly exploring high performance hardware features suitable for executing specialized trading strategies in order to efficiently trade at various market conditions.

We also presented a software architecture model that we used for implementing the proposed ATS system architecture, which employed a publish/subscribe communication model. The development of the modules of the ATS software architecture is an ongoing work and, in the current stage of development, we are able to run our ATS in back-test mode with moving-average crossover strategies on minute-by-minute databases. We achieved very satisfactory performance results in these experiments, processing 306.791 database rows representing more than two years of data in only 47 seconds.

Our future research works include evaluating the ATS system architecture performance in live mode, investigating more sophisticated trading strategies using artificial intelligence-based signal generators we have developed in previous works [5, 16], and evaluating the full capacity of our ATS by running it in distributed environments.

7. ACKNOWLEDGMENTS

We like to thank the partnership of PEIXE PILOTO INVESTIMENTOS trading firm for providing all the data sets for this research.

References

- [1] I. Aldridge. Measuring accuracy of trading strategies. *The Journal of Trading*, 4(3):17–25, Summer 2009.
- [2] I. Aldridge. *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Wiley, 2013.
- [3] R. Almgren and J. Lorenz. Bayesian adaptive trading with a daily cycle. *The Journal of Trading*, 1(4):38–46, Fall 2006.
- [4] R. W. Colby. *The encyclopedia of technical market indicators*. McGraw-Hill, New York, 2 edition, 2003.

- [5] A. F. De Souza, F. D. Freitas, and A. G. C. de Almeida. Fast learning and predicting of stock returns with virtual generalized random access memory weightless neural networks. *Concurrency and Computation: Practice and Experience*, 24(8):921–933, 2012.
- [6] F. J. Fabozzi, S. M. Focardi, and C. Jonas. *Challenges in Quantitative Equity Management (corrected July 2008)*. The Research Foundation of CFA Institute, Apr. 2008.
- [7] B. P. Gerkey and M. J. Matarić. Murdoch: publish/subscribe task allocation for heterogeneous agents. In *Proceedings of the fourth international conference on Autonomous agents, AGENTS '00*, pages 203–204, New York, NY, USA, 2000. ACM.
- [8] P. Gomber, B. Arndt, M. Lutat, and T. Uhle. High-frequency trading. Technical report, Deutsche Börse AG, 2011.
- [9] Google. Google trends results for research term "high frequency trading" — <http://www.google.com/trends/>, 2013.
- [10] T. Hendershott, C. M. Jones, and A. J. Menkveld. Does algorithmic trading improve liquidity? *Journal of Finance*, 66(1):1–33, Feb. 2011.
- [11] K. Hung, Y. Cheung, and L. Xu. An extended ASLD trading system to enhance portfolio management. *IEEE Transactions on Neural Networks*, 14(2):413–425, Mar. 2003.
- [12] M. Kearns, A. Kulesza, and Y. Nevmyvaka. Empirical limitations on high frequency trading profitability. Working Paper, 2010.
- [13] A. A. Kirilenko, A. S. Kyle, M. Samadi, and T. Tuzun. The flash crash: The impact of high frequency trading on an electronic market. Working Paper, 2011.
- [14] R. Kissell and M. Glantz. *Optimal Trading Strategies: Quantitative Approaches for Managing Market Impact and Trading Risk*. AMACOM, New York, USA, 2003.
- [15] A. Koors and B. Page. A hierarchical simulation based software architecture for back-testing and automated trading. In *Proceedings - 25th European Conference on Modelling and Simulation, ECMS 2011*, pages 275–282, Washington, DC, USA, 2011. IEEE Society Press.
- [16] C. Lee and J. Tzeng. Trend-oriented training for neural networks to forecast stock markets. *Asia Pacific Management Review*, 18(2):181–195, June 2013.
- [17] R. K. Narang. *Inside the Black Box: The Simple Truth About Quantitative Trading*. Wiley Finance, New York, 2 edition, 2013.
- [18] G. Nuti, M. Mirghaemi, P. Treleaven, and C. Yingsaeree. Algorithmic trading. *Computer*, 44:61–69, 2011.
- [19] R. Pardo. *The evaluation and optimization of trading strategies*. Wiley trading series, 2008.
- [20] S. Raudys. Portfolio of automated trading systems: Complexity and learning set size issues. *IEEE Transactions on Neural Networks and Learning Systems*, 24(3):448–459, 2013.
- [21] R. Simmons and D. James. *Inter Process Communication: A Reference Manual*. Robotics Institute, Carnegie Mellon University, Apr. 2011. IPC Version 3.9.
- [22] A. D. Wissner-Gross and C. E. Freer. Relativistic statistical arbitrage. *Phys. Rev. E*, 82:056104, Nov 2010.